

Experiment No.: 06

Title: Jenkins Pipeline using Domain Specific Language (DSL)

Batch: B-1**Roll No.: 16010422234****Experiment No.: 06**

Aim: To create and run Jenkins pipeline using Domain Specific Language (DSL)

Resources needed: Jenkins

Theory:

Jenkins Pipeline is a combination of plugins that supports integration and implementation of continuous delivery pipelines. It has an extensible automation server to create simple and complex delivery pipelines as code via pipeline Domain Specific Language (DSL). A Pipeline is a group of events interlinked with each other in a sequence.



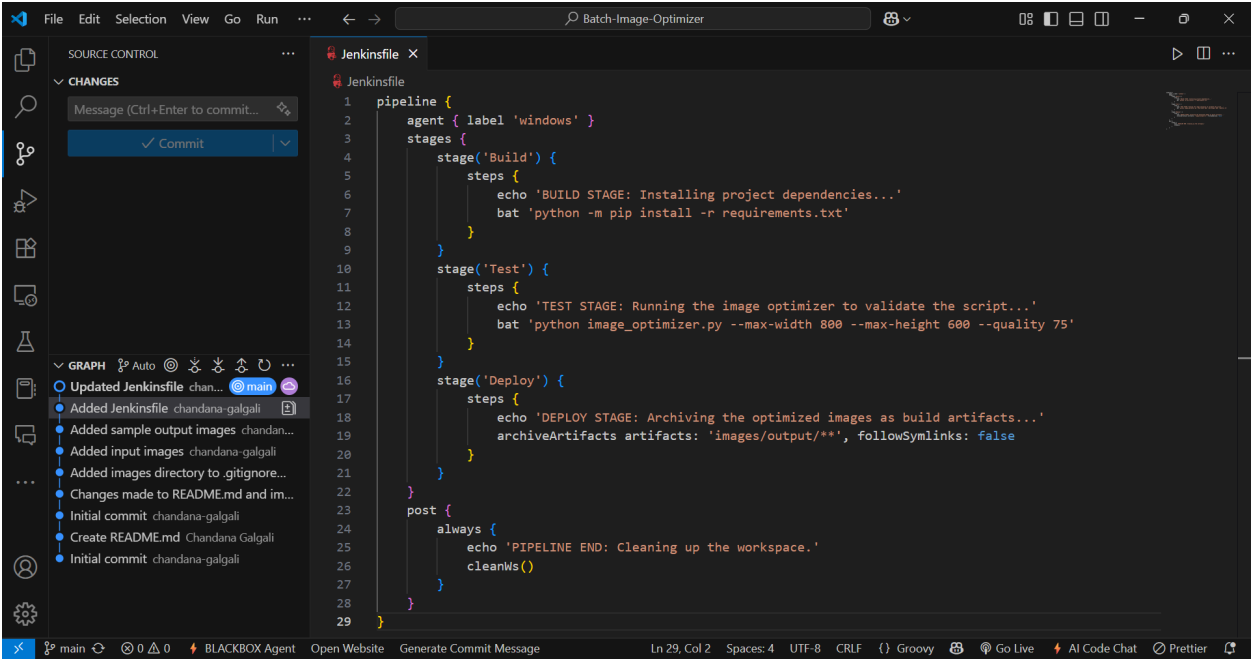
Jenkins job DSL is a plugin that allows us to define jobs in programmatic form with minimal effort. You can describe your jobs in Jenkins using a Groovy Based Language. It is similar to java but simpler because it's much more dynamic. It is Scripting Language. Jenkins job DSL plugin was designed to make it easier to manage jobs.

Procedure:

1. Install build pipeline and view pipeline
 2. Create sample jobs [example: build, test and Deploy].
 3. Create a pipeline for the sample script.
 4. Check the view pipeline to see the execution process.
-

Results: (Document with screenshots)

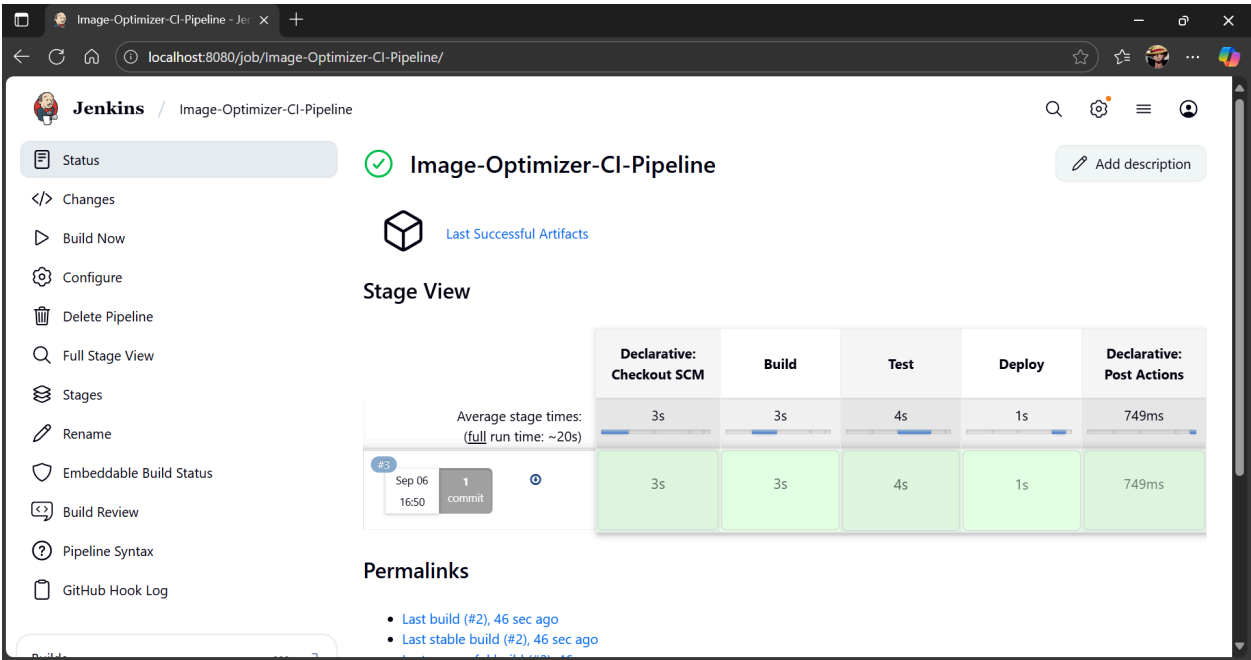
1. All steps to install, configure pipeline
2. View pipeline execution process



The screenshot shows the Visual Studio Code editor with a Jenkinsfile open. The left sidebar displays the 'SOURCE CONTROL' view with a commit history graph. The Jenkinsfile content is as follows:

```
1 pipeline {
2   agent { label 'windows' }
3   stages {
4     stage('Build') {
5       steps {
6         echo 'BUILD STAGE: Installing project dependencies...'
7         bat 'python -m pip install -r requirements.txt'
8       }
9     }
10    stage('Test') {
11      steps {
12        echo 'TEST STAGE: Running the image optimizer to validate the script...'
13        bat 'python image_optimizer.py --max-width 800 --max-height 600 --quality 75'
14      }
15    }
16    stage('Deploy') {
17      steps {
18        echo 'DEPLOY STAGE: Archiving the optimized images as build artifacts...'
19        archiveArtifacts artifacts: 'images/output/**', followSymlinks: false
20      }
21    }
22  }
23  post {
24    always {
25      echo 'PIPELINE END: Cleaning up the workspace.'
26      cleanWs()
27    }
28  }
29 }
```

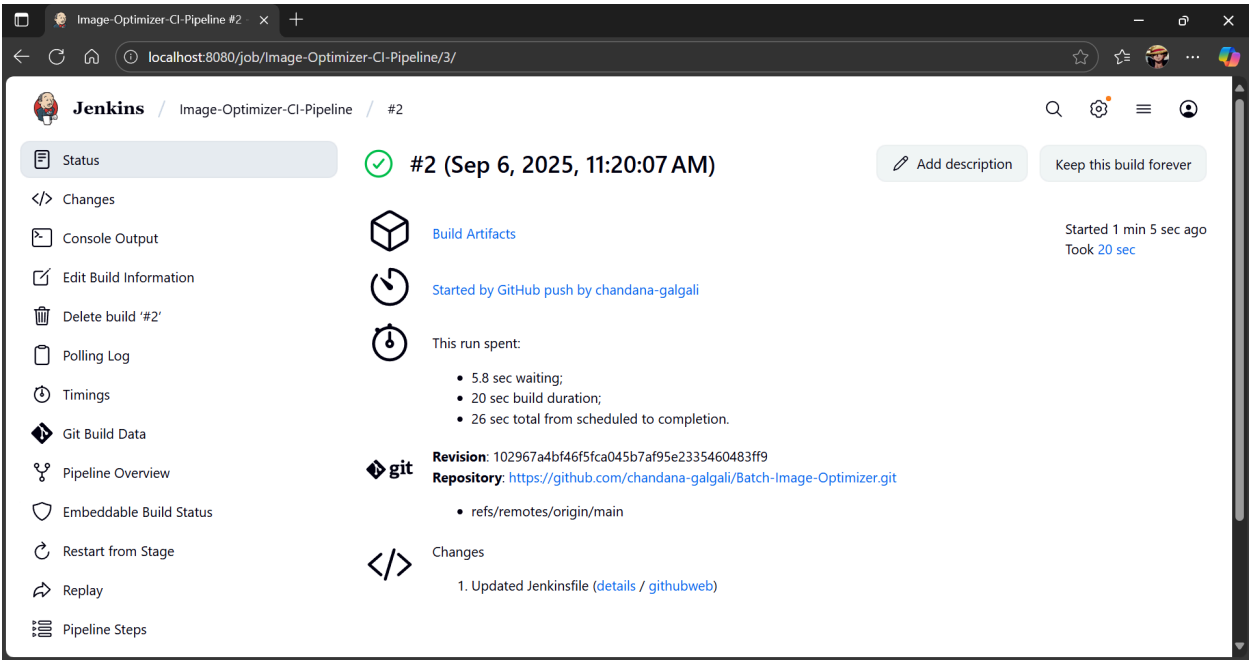
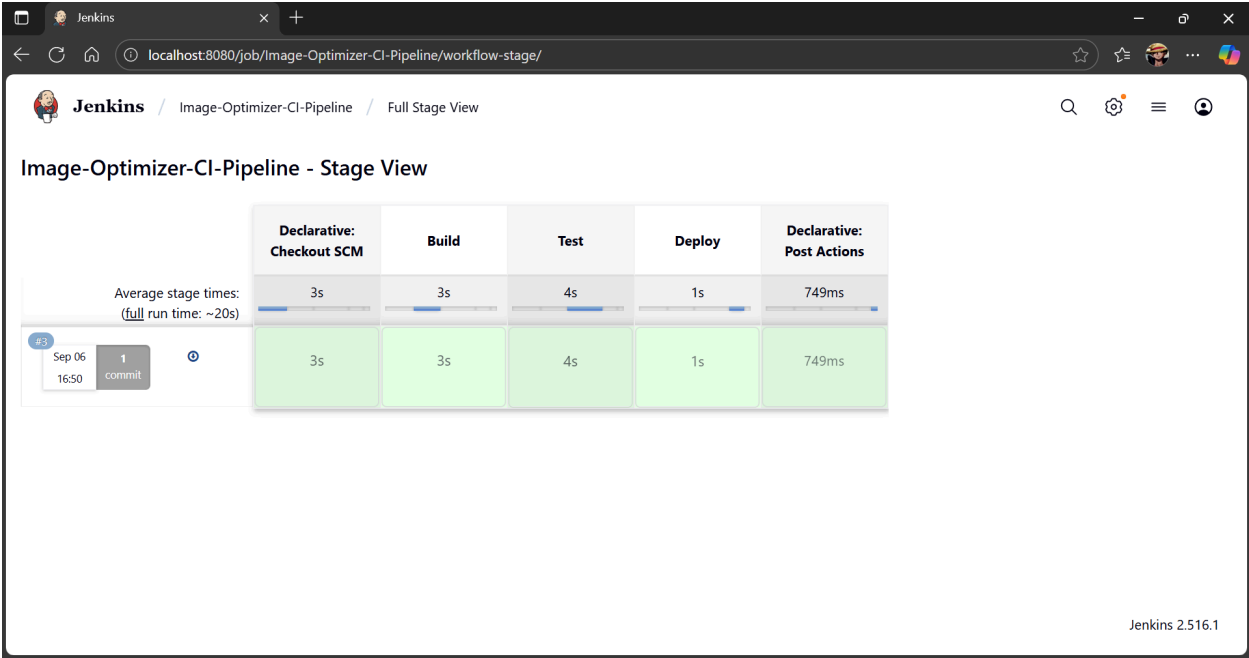
The commit history graph on the left shows a series of commits by 'chandana-galgali' on the 'main' branch, including updates to the Jenkinsfile, sample output images, input images, and the README.md file.



The screenshot shows the Jenkins web interface for the 'Image-Optimizer-CI-Pipeline'. The pipeline is in a successful state, indicated by a green checkmark. The 'Stage View' section displays a table of stage execution times for the last build (#3).

Stage	Declarative: Checkout SCM	Build	Test	Deploy	Declarative: Post Actions
Average stage times (full run time: ~20s)	3s	3s	4s	1s	749ms
#3 Sep 06 16:50 1 commit	3s	3s	4s	1s	749ms

The 'Permalinks' section provides links to the last build and the last stable build, both of which completed 46 seconds ago.



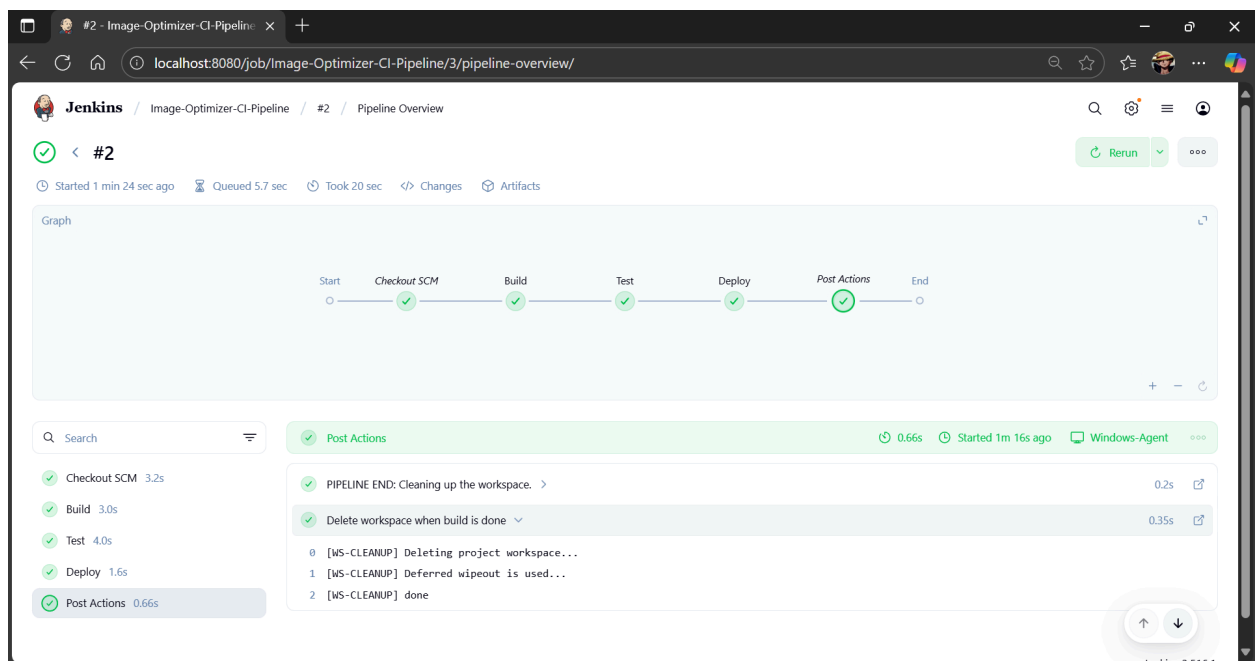
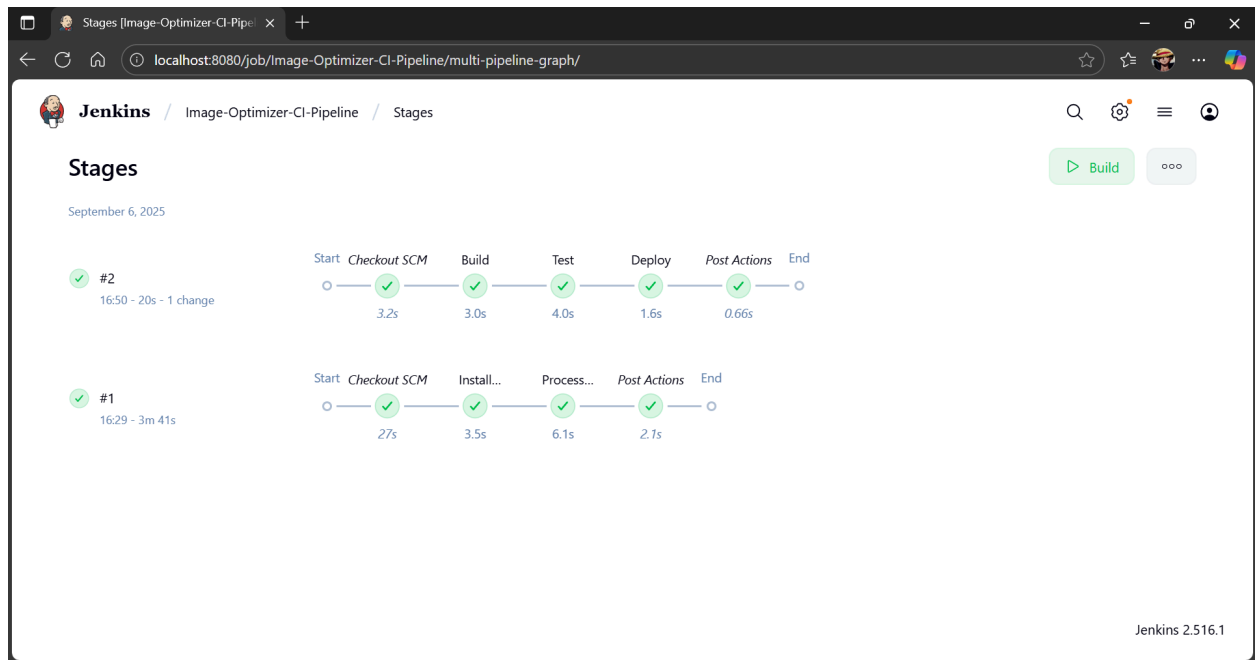


Image-Optimizer-CI-Pipeline #2

localhost:8080/job/Image-Optimizer-CI-Pipeline/3/flowGraphTable/

Jenkins

Image-Optimizer-CI-Pipeline / #2 / Pipeline Steps

Status

Changes

Console Output

Edit Build Information

Delete build '#2'

Polling Log

Timings

Git Build Data

Pipeline Overview

Embeddable Build Status

Restart from Stage

Replay

Pipeline Steps

Step	Arguments	Status
Start of Pipeline - (15 sec in block)		✓
node - (13 sec in block)	windows	✓
node block - (13 sec in block)		✓
stage - (3.3 sec in block)	Declarative: Checkout SCM	✓
stage block (Declarative: Checkout SCM) - (3.2 sec in block)		✓
checkout - (3.1 sec in self)		✓
withEnv - (10 sec in block)	GIT_BRANCH, GIT_COMMIT, GIT_PREVIOUS_COMMIT, GIT_PREVIOUS_SUCCESSFUL_COMMIT, GIT_URL	✓
withEnv block - (10 sec in block)		✓
stage - (3.1 sec in block)	Build	✓

Image-Optimizer-CI-Pipeline #2

localhost:8080/job/Image-Optimizer-CI-Pipeline/3/flowGraphTable/

Jenkins

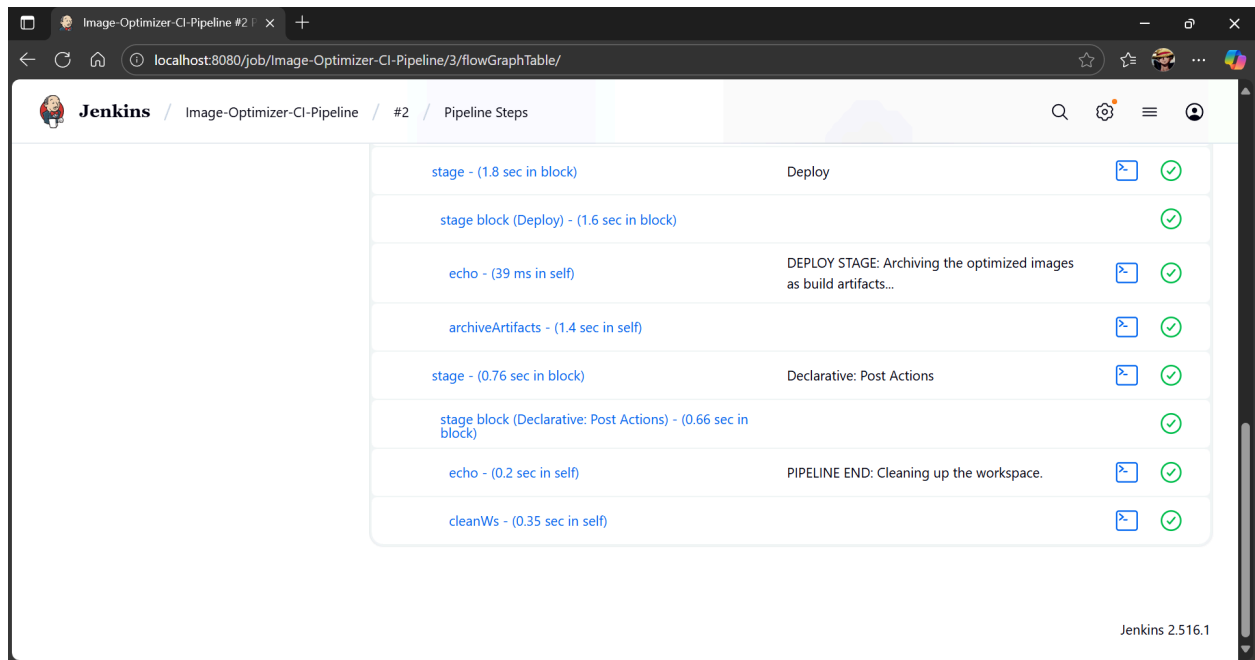
Image-Optimizer-CI-Pipeline / #2 / Pipeline Steps

Pipeline Steps

Workspaces

Previous Build

stage - (3.1 sec in block)	Build	✓
stage block (Build) - (3 sec in block)		✓
echo - (38 ms in self)	BUILD STAGE: Installing project dependencies...	✓
bat - (2.8 sec in self)	python -m pip install -r requirements.txt	✓
stage - (4.1 sec in block)	Test	✓
stage block (Test) - (4 sec in block)		✓
echo - (43 ms in self)	TEST STAGE: Running the image optimizer to validate the script...	✓
bat - (3.8 sec in self)	python image_optimizer.py --max-width 800 --max-height 600 --quality 75	✓
stage - (1.8 sec in block)	Deploy	✓
stage block (Deploy) - (1.6 sec in block)		✓



Questions:

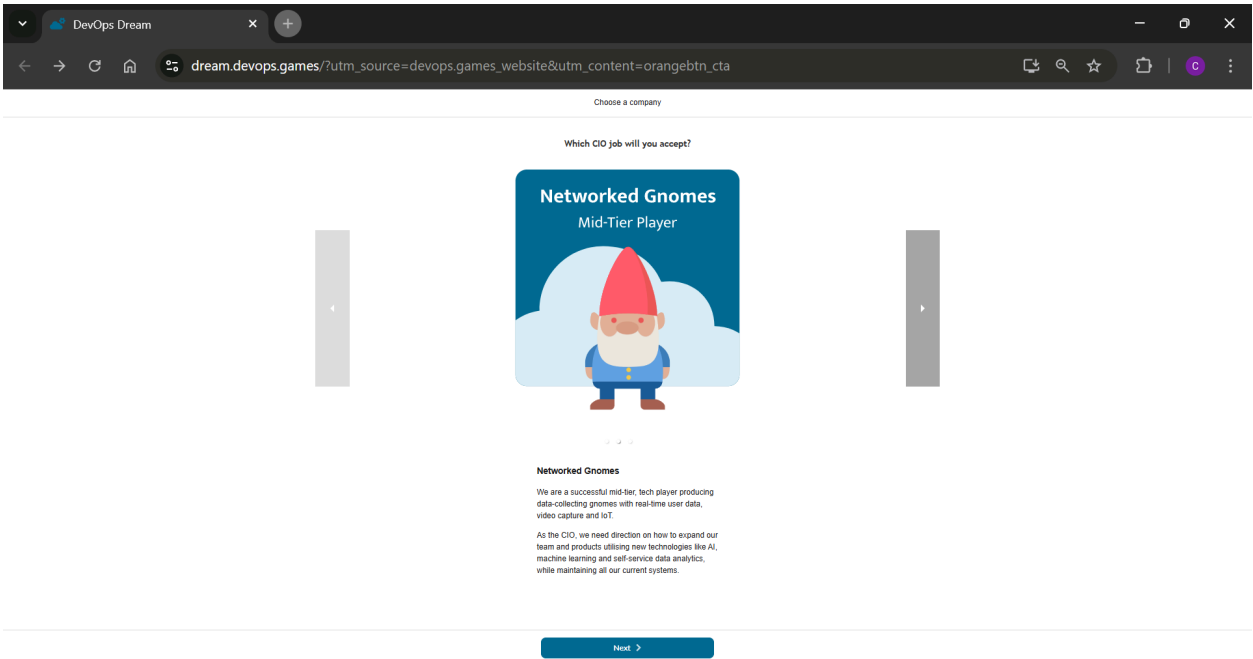
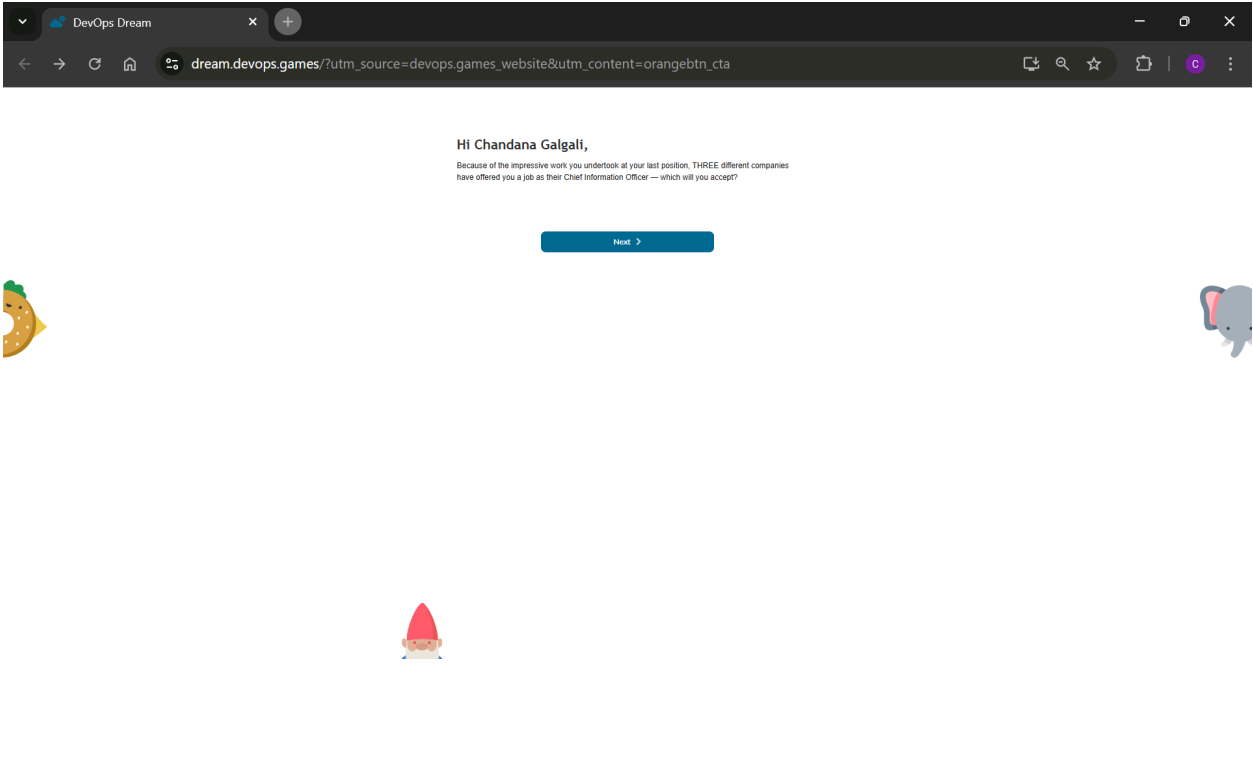
1. Explore “DevOpsDream game on <https://devops.games>”. DevOpsDream built as a tool for learning about what works and doesn’t work in the world of software development. The DevOpsDream is an attempt to inspire you to think about how you develop and run software. Read the instructions and play the game. Prepare a summary sheet for the same. (Understanding of concept, Snapshot of best result)

Ans: The "DevOpsDream" game is an interactive simulation that brilliantly gamifies the challenges of managing a software development project. The player takes on the role of a project lead, starting with a small team and limited features. The core goal is to deliver features to production to earn money, which can then be reinvested into hiring more developers, improving processes (like adding CI/CD or automated testing), and managing technical debt.

The game forces the player to constantly balance competing priorities:

- Feature Velocity: Should you hire more developers to release features faster and earn more money?
- Quality & Stability: Should you invest in testing and CI/D pipelines to reduce bugs and prevent production failures?
- Technical Debt: Should you spend time refactoring old code to improve long-term stability, even if it means slowing down new feature development?

Making the wrong decisions leads to cascading failures: production goes down, money is lost, developers get stressed and quit, and the project grinds to a halt. Making the right decisions leads to a stable, profitable, and efficient development lifecycle. It's a powerful and practical demonstration of core DevOps principles in action.



DevOps Dream

dream.devops.games/?utm_source=devops.games_website&utm_content=orangebtn_cta

Welcome CIO



Welcome to **Networked Gnomes**, Chandana Galgali!

We're excited to have you as our new **Chief Information Officer**!

As you know, we've been **running for 7 years** and our biggest problem is knowing what to tackle next."

We want to expand our team and products with new technologies like AI, machine learning and self-service data analytics –

but we're not sure how to set up our internal teams to succeed. They have their hands full running the current systems.

How do we keep the lights on for current customers while developing new products and solutions to attract new business in the market?

To move our company forward, you have to invest in some **Initiatives**.



Next >

DevOps Dream

dream.devops.games/?utm_source=devops.games_website&utm_content=orangebtn_cta

Your Starting Report

Q1

Q2

Q3

Q4

Time 1



Chandana Galgali
CIO, Networked Gnomes

▼ Performance Metrics

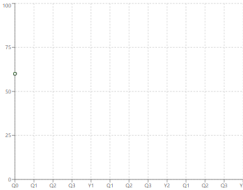
People

Satisfaction

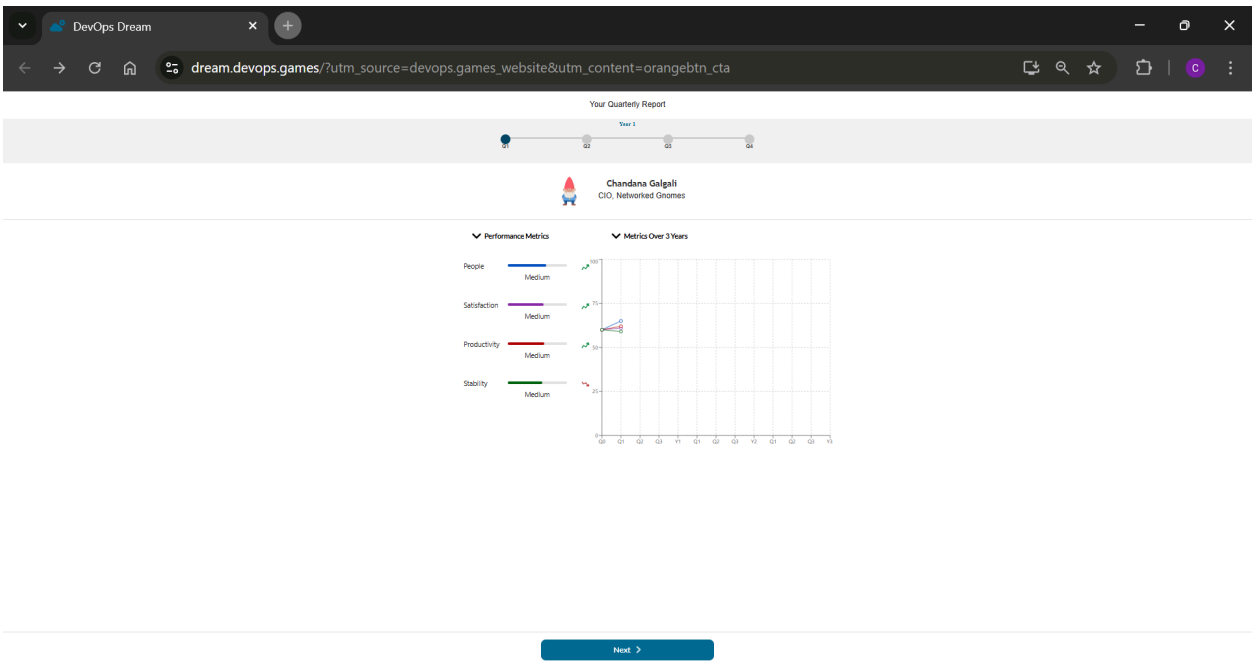
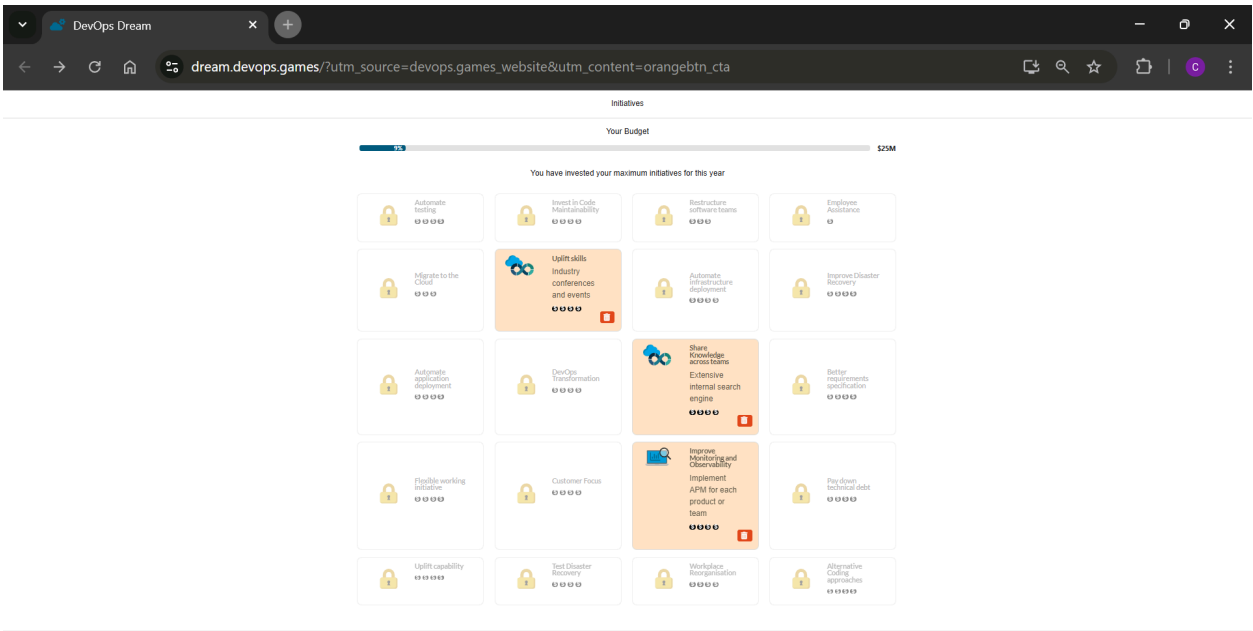
Productivity

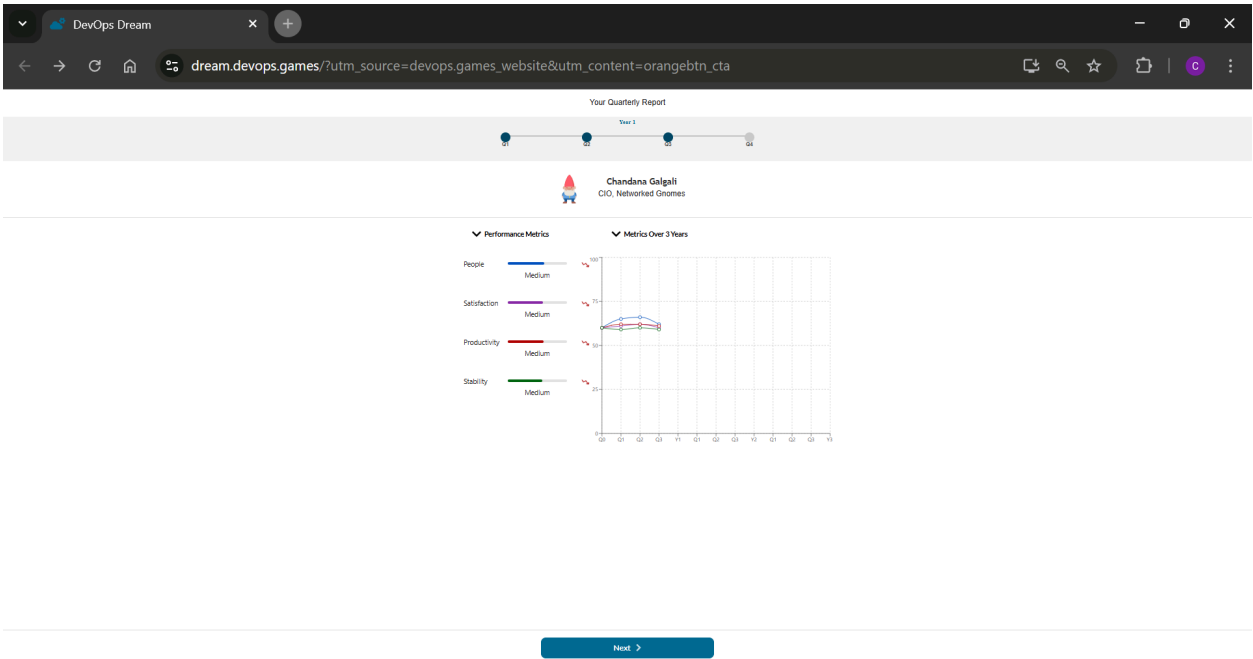
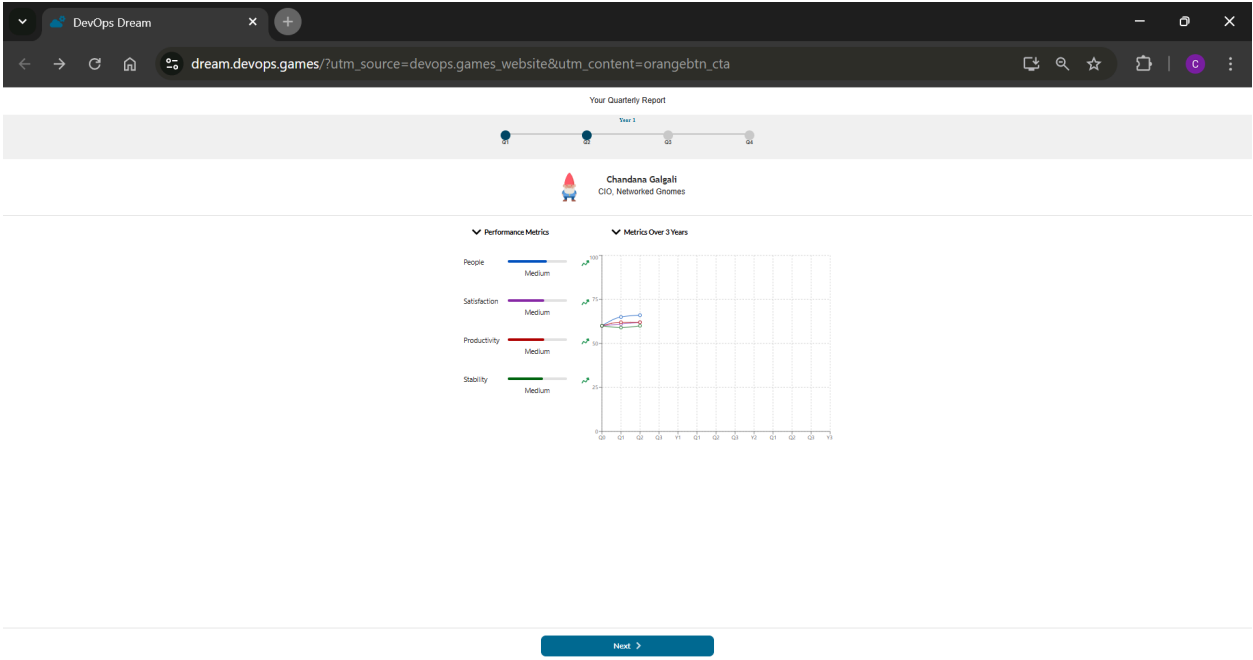
Stability

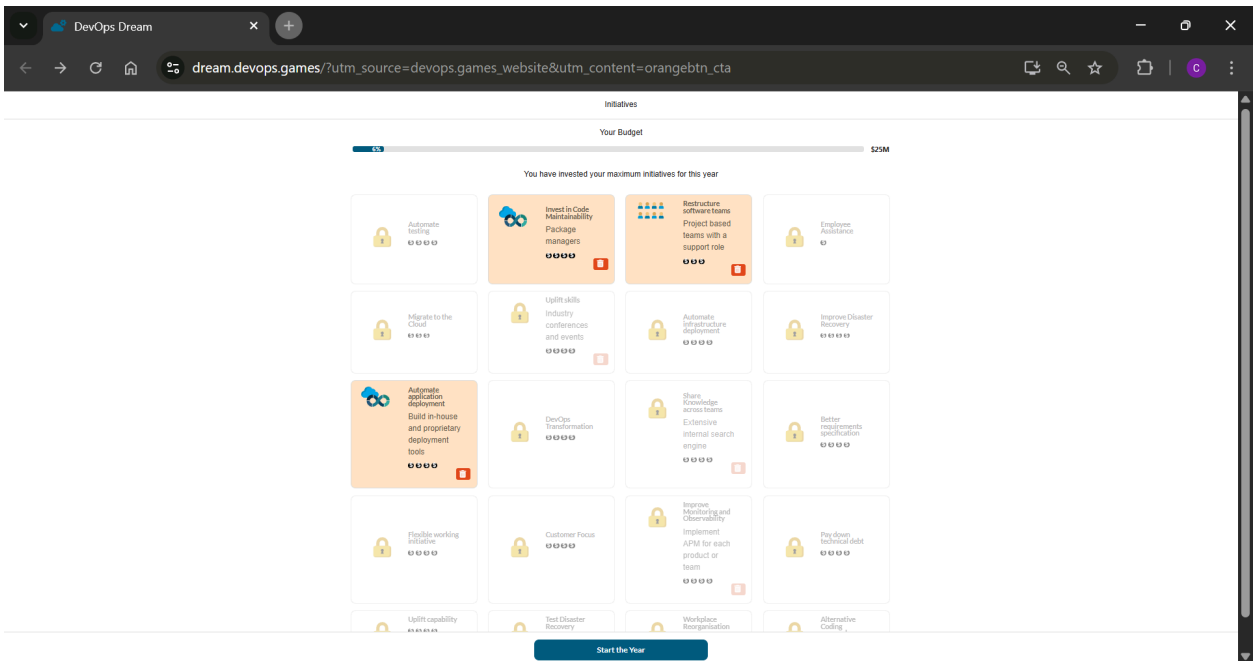
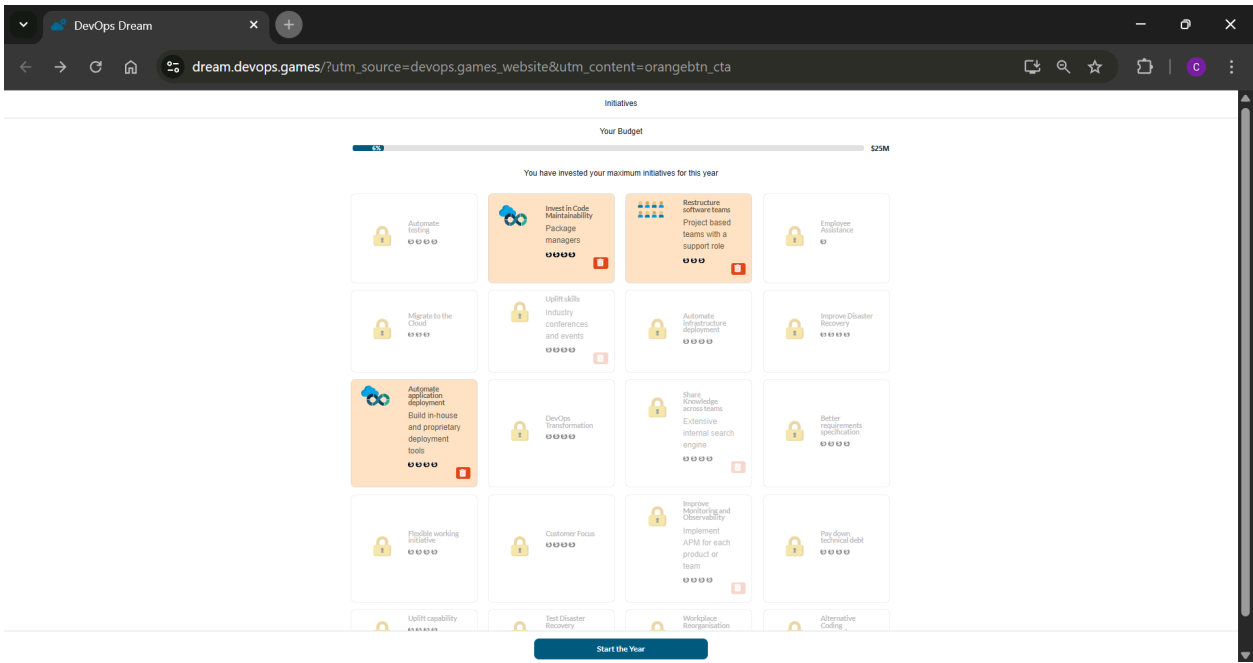
▼ Metrics Over 3 Years

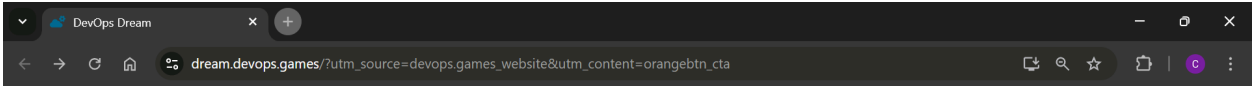


Next >







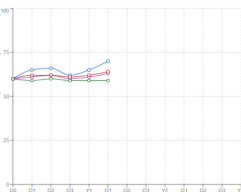


Chandana Galgali
CIO, Networked Gnomes

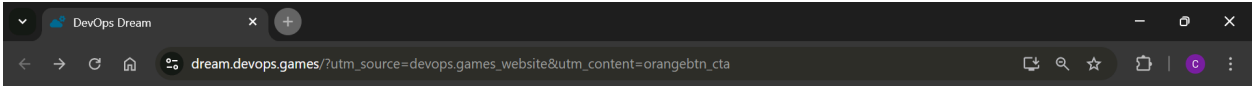
Performance Metrics



Metrics Over 3 Years



Next >

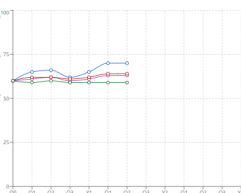


Chandana Galgali
CIO, Networked Gnomes

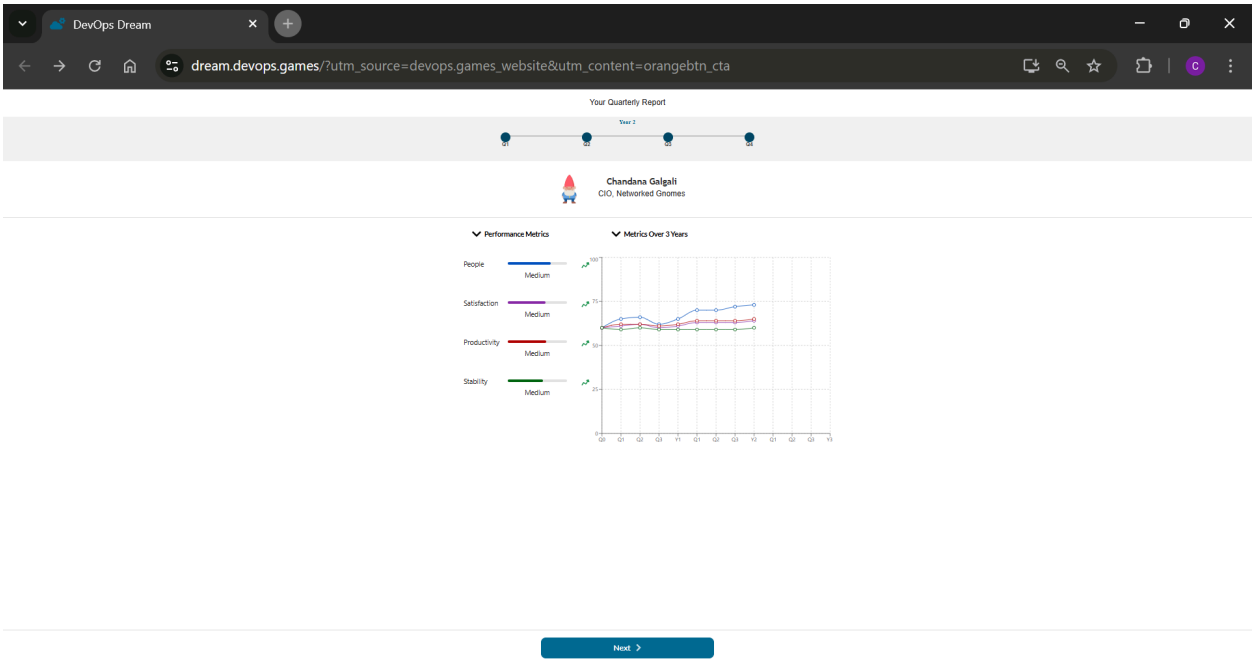
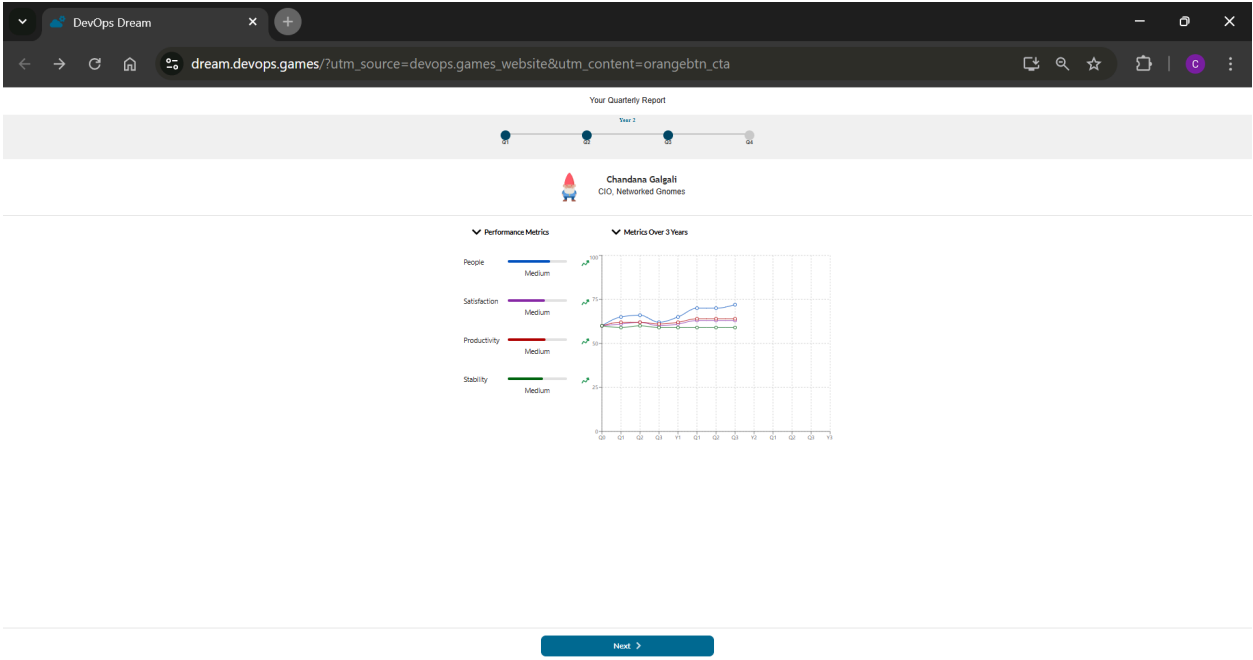
Performance Metrics

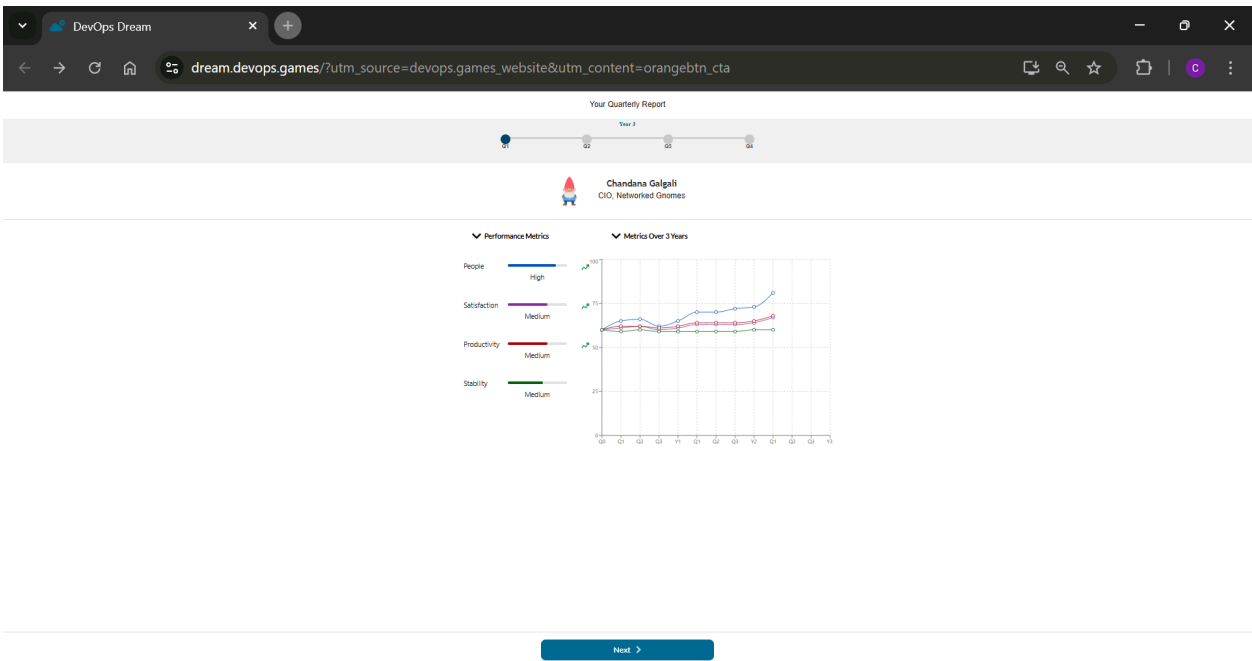
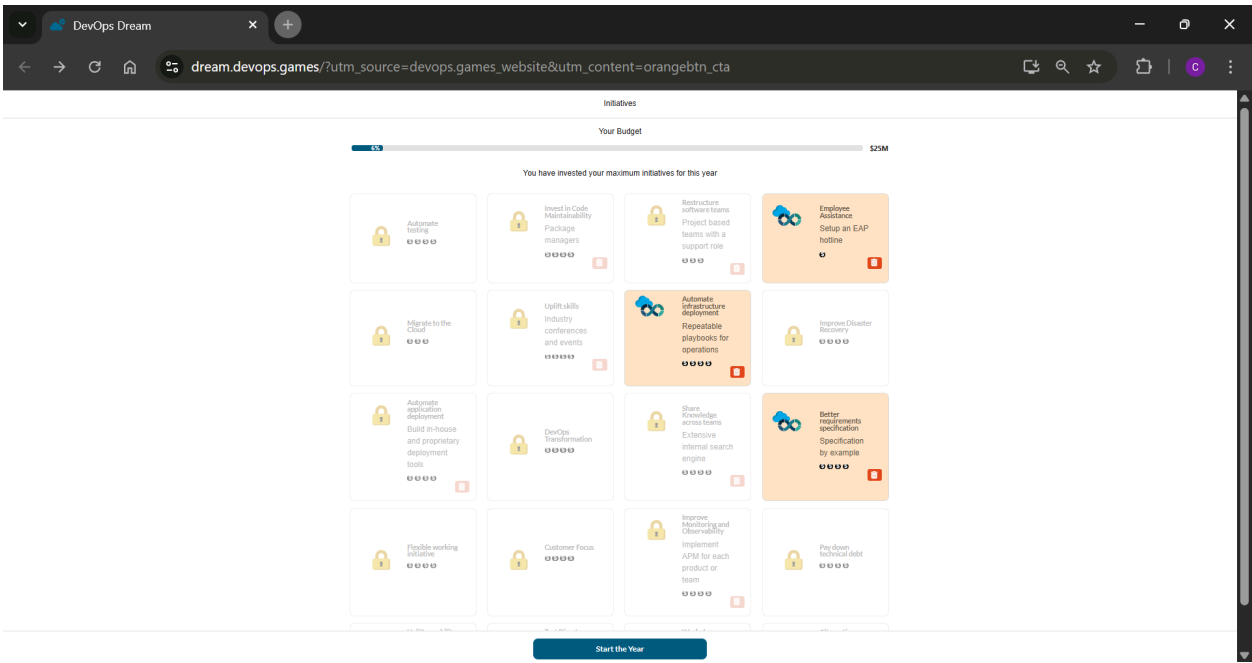


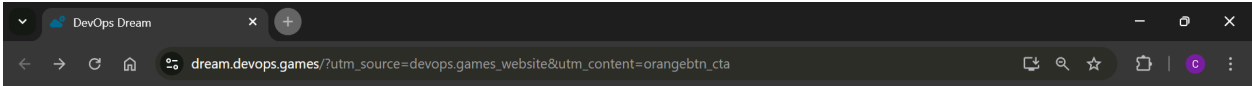
Metrics Over 3 Years



Next >





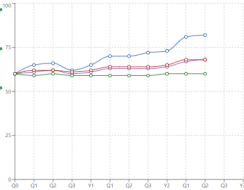


Chandana Galgali
CIO, Networked Gnomes

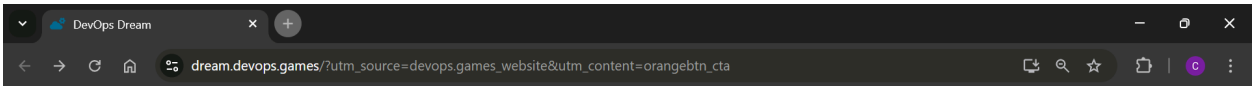
Performance Metrics



Metrics Over 3 Years



Next >

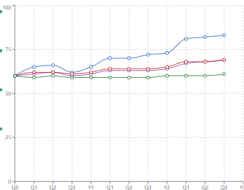


Chandana Galgali
CIO, Networked Gnomes

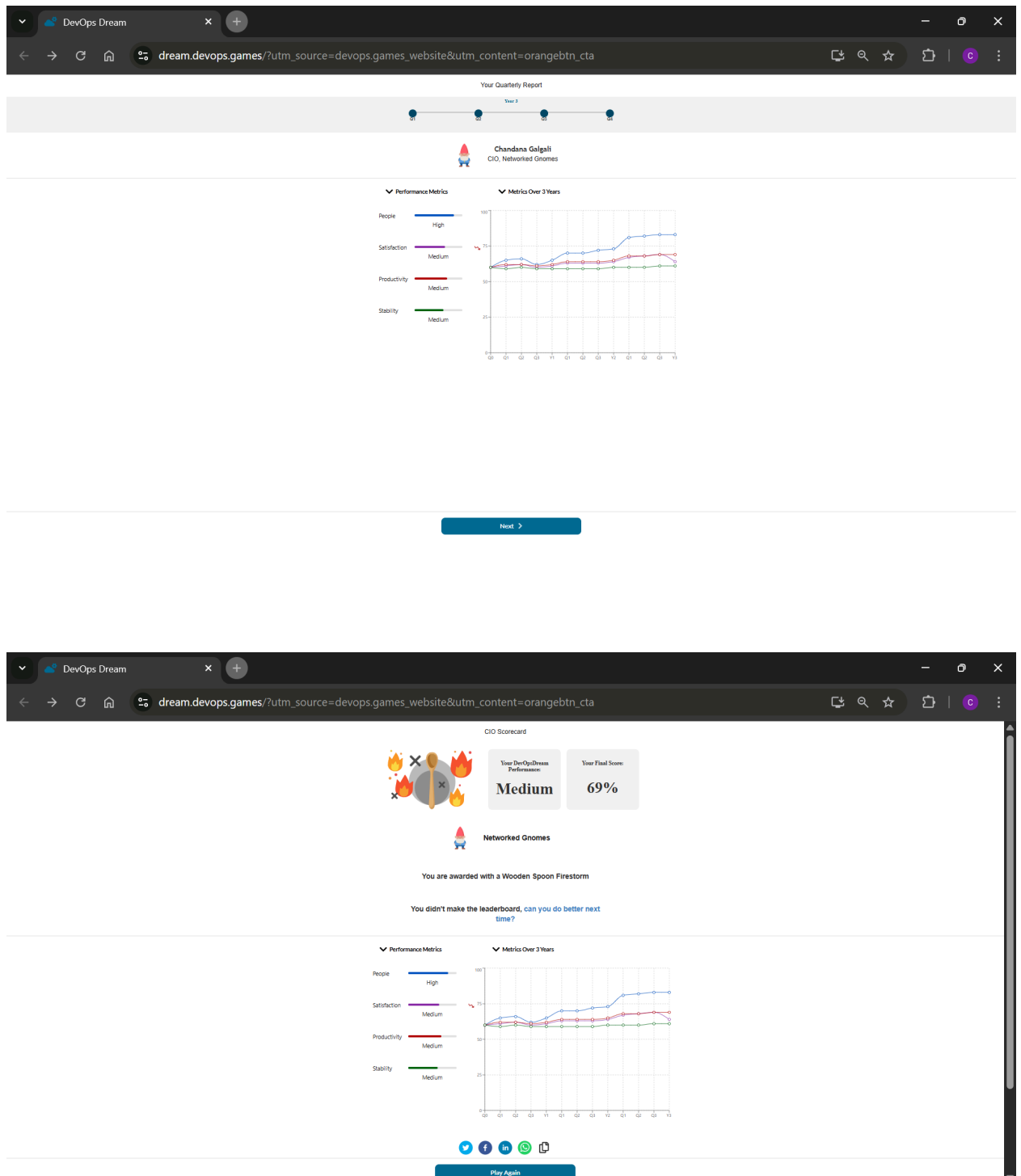
Performance Metrics



Metrics Over 3 Years



Next >



2. How game based learning helps in understanding concept, logic and decision making?

Ans: Game-based learning, as demonstrated by "DevOpsDream," is highly effective for understanding complex concepts like DevOps for several key reasons:

- **Safe Environment for Failure:** Games provide a consequence-free sandbox. You can make a terrible decision, like hiring ten developers without any testing infrastructure, and see the entire project collapse. This failure is a powerful learning experience that you couldn't afford in the real world. You learn the why behind best practices by experiencing the negative outcomes of ignoring them.

- Immediate Feedback Loops: In a game, the consequences of your decisions are immediate. When you invest in a CI/CD pipeline, you instantly see the rate of bugs go down. When you ignore technical debt, you see production failures increase. This rapid cause-and-effect relationship makes abstract concepts tangible and easy to grasp.
- Active Engagement vs. Passive Learning: Instead of just reading about the benefits of CI/CD, the game forces you to actively decide when and how to implement it. This active participation engages critical thinking and problem-solving skills, leading to deeper and more memorable learning than simply reading a textbook or watching a lecture.
- Simplifies Complex Systems: A real-world DevOps environment has thousands of variables. A game like DevOpsDream simplifies this into a manageable model with a few key metrics (money, bugs, features, stability). This allows the player to focus on the core logic and strategic decision-making without being overwhelmed by technical details, making it easier to understand the fundamental principles at play.

Outcomes: CO3 — Comprehend the effective code building and testing process

Conclusion: (Conclusion to be based on the Results and outcomes achieved)

This experiment used Jenkins' Declarative DSL to create a "Pipeline as Code" by defining a multi-stage workflow (Build, Test, Deploy) in a Jenkinsfile. The Stage View provided clear visual feedback of the build's flow. This demonstrates a core DevOps practice for creating transparent and reproducible CI/CD pipelines.

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of faculty in-charge with date

References:

Books/ Journals/ Websites:

1. Jenkins Pipeline Tutorial: <https://www.guru99.com/jenkins-pipeline-tutorial.html>
 2. Part1-SDET Essentials | Jenkins Build & Delivery Pipeline | CI & CD | How to create Jenkins Pipeline: https://www.youtube.com/watch?v=1ji_9scA2C4
 3. Part2-SDET Essentials| CI & CD|How to Build Pipelines with Groovy Script|Scripted & Declarative: <https://www.youtube.com/watch?v=Ozna0MS5D1E>
 4. Jenkins Pipeline Tutorial | CI/CD Pipeline Jenkins | Jenkins Tutorial | DevOps Training Edureka: https://www.youtube.com/watch?v=_fmX31VFbL8&t=1796s
 5. <https://devops.games>
-